

第1章-4: Jetpack Compose 核心UI要素 (小白专属)

讲师寄语: 各位小伙伴, 上一章咱们入门了Compose的基本概念, 今天咱们聚焦UI搭建的"三大基石"——Column布局、修饰符和UI美化要素(颜色、排版、形状)。这部分内容是你以后写任何Compose界面都离不开的, 咱们依旧用"代码+讲解"的方式, 边敲边理解, 保证不枯燥!

一、先稳根基: Column 垂直布局详解

上一章咱们简单提过Column是"垂直布局容器", 就像家里的"书架", 只能把书(UI元素)从上到下依次摆放。今天咱们把它讲透, 包括如何控制元素位置、间距和换行问题。

1. Column的核心作用: 管理垂直排列的元素

先看一个基础示例, 感受Column的基本用法:

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.Text
3  import androidx.compose.runtime.Composable
4  import androidx.compose.ui.Modifier
5  import androidx.compose.ui.unit.dp
6  import androidx.compose.foundation.layout.padding
7
8  @Composable
9  fun BasicColumnDemo() {
10     // 最基础的Column: 子元素默认从上到下、左对齐
11     Column(
12         modifier = Modifier.padding(16.dp) // 给整个Column加内边距
13     ) {
14         Text(text = "书架第一层: 语文书", modifier = Modifier.padding(4.dp))
15         Text(text = "书架第二层: 数学书", modifier = Modifier.padding(4.dp))
16         Text(text = "书架第三层: 英语书", modifier = Modifier.padding(4.dp))
17     }
18 }
```

运行后你会看到三行文字垂直排列, 这就是Column的核心能力——约束子元素的排列方向为垂直。

2. 关键参数: 控制元素的位置和间距

Column有两个超重要的参数，能帮你精准控制子元素的布局，咱们逐个击破：

(1) verticalArrangement：控制子元素的垂直分布

这个参数决定了子元素在Column的"垂直空间"里如何分布，比如居中、靠顶部还是均匀分散。常用值有3个：

- **Arrangement.Top**：子元素靠顶部排列（默认值）
- **Arrangement.Center**：子元素在Column中垂直居中
- **Arrangement.SpaceBetween**：子元素均匀分布，首尾元素贴边，中间元素间距相等

示例：让子元素垂直居中并均匀分散

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.foundation.layout.verticalArrangement
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.Composable
6  import androidx.compose.ui.Modifier
7
8  @Composable
9  fun ColumnArrangementDemo() {
10     Column(
11         modifier = Modifier.fillMaxSize(), // 让Column占满整个屏幕高度
12         verticalArrangement = Arrangement.SpaceBetween // 子元素垂直均匀分散
13     ) {
14         Text(text = "顶部元素")
15         Text(text = "中间元素")
16         Text(text = "底部元素")
17     }
18 }
```

运行后会发现，三个文字分别贴在屏幕顶部、中间和底部，这就是verticalArrangement的作用。

(2) horizontalAlignment：控制子元素的水平对齐方式

这个参数决定了子元素在Column的"水平方向"上的对齐位置，比如左对齐、居中、右对齐。常用值：

- **Alignment.Start**：左对齐（默认值，适合中文）
- **Alignment.CenterHorizontally**：水平居中
- **Alignment.End**：右对齐

示例：子元素水平居中+垂直居中

```

1  @Composable
2  fun ColumnAlignmentDemo() {
3      Column(
4          modifier = Modifier.fillMaxSize(),
5          verticalArrangement = Arrangement.Center, // 垂直居中
6          horizontalAlignment = Alignment.CenterHorizontally // 水平居中
7      ) {
8          Text(text = "我在屏幕正中间")
9          Text(text = "我也在正中间")
10     }
11 }

```

 小技巧：如果想让某个子元素打破Column的统一对齐（比如其他元素居中，就它右对齐），可以给这个子元素的Modifier加**align(Alignment.End)**，示例：

```

Column(...) {
    Text(text = "水平居中")
    Text(text = "我要右对齐", modifier =
Modifier.align(Alignment.End))
}

```

二、UI的"化妆师"：修饰符（Modifier）全解析

修饰符就像UI元素的"化妆品和工具"，可以给按钮、文字、布局添加边框、背景、间距、大小等属性。没有修饰符的UI元素就像"素颜"，有了修饰符才能变得"精致"。

1. 修饰符的核心特点：链式调用

Compose的修饰符支持"链式调用"，就是用点（.）把多个修饰符连起来，写法简洁又直观。比如：

Code block

```

1  Modifier
2      .fillMaxWidth() // 第一步：占满父容器宽度
3      .padding(16.dp) // 第二步：加内边距
4      .background(Color.LightGray) // 第三步：加灰色背景
5      .border(2.dp, Color.Black) // 第四步：加黑色边框

```

注意：修饰符的调用顺序会影响效果，比如先加padding再加background，背景会在padding内侧；先加background再加padding，背景会包含padding区域，大家可以自己动手试一下。

2. 常用修饰符：小白必掌握的10个

咱们不用记所有修饰符，先掌握以下最常用的，就能满足80%的需求：

修饰符	功能描述	示例
fillMaxWidth()	占满父容器的宽度	Modifier.fillMaxWidth()
fillMaxHeight()	占满父容器的高度	Modifier.fillMaxHeight()
fillMaxSize()	占满父容器的宽和高（常用）	Modifier.fillMaxSize()
padding()	添加内边距（元素内部和内容 的距离）	Modifier.padding(16.dp) 或 padding(8.dp,16.dp)
margin()	添加外边距（元素和其他元素 的距离）	Modifier.margin(8.dp)
background()	设置背景颜色或背景图	Modifier.background(Color. Red)
border()	设置边框（宽度+颜色+形 状）	Modifier.border(2.dp, Color.Black)
size()	设置固定宽高	Modifier.size(100.dp) 或 size(80.dp, 120.dp)
width()/height()	单独设置宽或高	Modifier.width(200.dp)
clickable()	给元素添加点击事件	Modifier.clickable { println("被点击了")} }

3. 实战：用修饰符打造一个精致按钮

把上面的修饰符组合起来，做一个美观的按钮：

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.foundation.layout.padding
4  import androidx.compose.material3.Button
5  import androidx.compose.material3.Text
6  import androidx.compose.runtime.Composable
7  import androidx.compose.ui.Alignment
8  import androidx.compose.ui.Modifier
9  import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.unit.dp
11
12 @Composable
13 fun StyledButtonDemo() {
14     Column(
```

```

15         modifier = Modifier.fillMaxSize(),
16         verticalArrangement = Arrangement.Center,
17         horizontalAlignment = Alignment.CenterHorizontally
18     ) {
19         Button(
20             onClick = { println("精致按钮被点击") },
21             modifier = Modifier
22                 .size(200.dp, 60.dp) // 宽200dp, 高60dp
23                 .padding(4.dp) // 内边距4dp
24                 .margin(16.dp) // 外边距16dp
25         ) {
26             Text(
27                 text = "精致按钮",
28                 color = Color.White // 文字白色
29             )
30         }
31     }
32 }

```

运行后你会看到一个大小合适、有间距的按钮，这就是修饰符的强大之处！

三、让UI变美：颜色、排版、形状

如果说Column和修饰符是UI的"骨架"，那颜色、排版、形状就是UI的"颜值"。咱们逐个学习如何用Compose设置这些属性。

1. 颜色（Color）：给UI穿件彩色衣服

Compose内置了大量系统颜色，也支持自定义颜色，常用的设置方式有3种：

（1）使用系统预设颜色

系统颜色在`androidx.compose.ui.graphics.Color`类中，直接调用即可，比如`Color.Red`、`Color.Blue`：

Code block

```

1  Text(
2      text = "红色文字",
3      color = Color.Red // 系统预设红色
4  )

```

（2）使用Material 3主题颜色

Compose默认支持Material 3（MD3）主题，主题颜色会统一APP风格，比如主题色（primary）、次要色（secondary）：

Code block

```
1 import androidx.compose.material3.MaterialTheme
2 import androidx.compose.material3.Text
3
4 @Composable
5 fun ThemeColorDemo() {
6     Text(
7         text = "主题主色文字",
8         color = MaterialTheme.colorScheme.primary // 主题主色
9     )
10    Text(
11        text = "主题次要色文字",
12        color = MaterialTheme.colorScheme.secondary // 主题次要色
13    )
14 }
```

好处：修改主题时，所有用了主题颜色的元素会自动同步，不用逐个修改。

(3) 自定义颜色

用RGB或十六进制值定义自己的颜色，比如微信绿、淘宝橙：

Code block

```
1 import androidx.compose.ui.graphics.Color
2
3 // 自定义颜色：RGB值 (0-255)
4 val WeChatGreen = Color(76, 217, 100)
5 // 自定义颜色：十六进制 (和XML里的#FF0000类似，这里去掉#，前面加0xFF)
6 val TaobaoOrange = Color(0xFFFF7A45)
7
8 @Composable
9 fun CustomColorDemo() {
10     Text(text = "微信绿", color = WeChatGreen)
11     Text(text = "淘宝橙", color = TaobaoOrange)
12 }
```

2. 排版 (Typography)：让文字更有层次感

排版就是控制文字的"样式"，比如字体大小、粗细、行高，好的排版能让界面更易读。Compose同样支持系统主题排版和自定义排版。

(1) 使用主题预设排版

MD3主题预设了不同场景的文字样式，比如标题、正文、副标题，直接调用即可：

```

1 import androidx.compose.material3.MaterialTheme
2 import androidx.compose.material3.Text
3 import androidx.compose.foundation.layout.Column
4 import androidx.compose.foundation.layout.padding
5 import androidx.compose.runtime.Composable
6
7 @Composable
8 fun ThemeTypographyDemo() {
9     Column(modifier = Modifier.padding(16.dp)) {
10         // 大标题
11         Text(
12             text = "这是大标题",
13             style = MaterialTheme.typography.headlineLarge
14         )
15         // 正文
16         Text(
17             text = "这是正文内容，字体大小适中，适合阅读。",
18             style = MaterialTheme.typography.bodyLarge
19         )
20         // 小标题
21         Text(
22             text = "这是小标题",
23             style = MaterialTheme.typography.titleMedium
24         )
25     }
26 }

```

(2) 自定义文字样式

如果预设样式不符合需求，用**TextStyle**自定义字体大小、粗细等：

Code block

```

1 import androidx.compose.ui.text.TextStyle
2 import androidx.compose.ui.text.font.FontWeight
3 import androidx.compose.ui.unit.sp
4
5 @Composable
6 fun CustomTypographyDemo() {
7     Text(
8         text = "自定义粗体文字",
9         style = TextStyle(
10             fontSize = 20.sp, // 字体大小20sp (sp是文字专用单位，适配字体缩放)
11             fontWeight = FontWeight.Bold, // 粗体
12             color = Color.Black, // 文字颜色
13             lineHeight = 30.sp // 行高30sp
14         )
15     )
16 }

```

```
15     )
16 }
```

 单位注意：文字大小用**sp**，不是dp！sp会跟随系统字体大小设置变化，保证用户能看清文字，这是Android开发的规范。

3. 形状（Shape）：给UI换个轮廓

默认的UI元素都是方形的，用Shape可以把它们变成圆形、圆角矩形等，让界面更柔和。Compose同样提供了预设形状和自定义形状。

（1）使用预设形状

预设形状在`androidx.compose.foundation.shape`中，常用的有：

- **CircleShape**：圆形
- **RoundedCornerShape(xx.dp)**：圆角矩形（xx是圆角大小）
- **CutCornerShape(xx.dp)**：切角矩形

实战：做一个圆形头像和圆角按钮

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.size
3  import androidx.compose.foundation.shape.CircleShape
4  import androidx.compose.foundation.shape.RoundedCornerShape
5  import androidx.compose.material3.Button
6  import androidx.compose.material3.Text
7  import androidx.compose.runtime.Composable
8  import androidx.compose.ui.Modifier
9  import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.unit.dp
11
12 @Composable
13 fun ShapeDemo() {
14     Column(modifier = Modifier.padding(16.dp)) {
15         // 圆形"头像"（用Text模拟，实际开发中用Image）
16         Text(
17             text = "头像",
18             modifier = Modifier
19                 .size(80.dp)
20                 .background(Color.Blue, shape = CircleShape) // 圆形背景
21                 .padding(8.dp),
22             color = Color.White
```

```

23         )
24
25         // 圆角按钮 (圆角16dp)
26         Button(
27             onClick = {},
28             modifier = Modifier.size(150.dp, 50.dp).margin(top = 16.dp),
29             shape = RoundedCornerShape(16.dp) // 圆角形状
30         ) {
31             Text(text = "圆角按钮")
32         }
33     }
34 }

```

(2) 自定义形状

如果需要更复杂的形状（比如五角星、三角形），可以自定义Shape，不过小白阶段先掌握预设形状即可，后续进阶再学自定义。

四、综合实战：搭建一个个人信息卡片

把今天学的Column、修饰符、颜色、排版、形状整合起来，做一个完整的个人信息卡片，巩固所有知识点：

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.Spacer
3  import androidx.compose.foundation.layout.fillMaxWidth
4  import androidx.compose.foundation.layout.height
5  import androidx.compose.foundation.layout.padding
6  import androidx.compose.foundation.layout.size
7  import androidx.compose.foundation.shape.CircleShape
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.material3.Card
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.text.TextStyle
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19
20 @Composable
21 fun UserInfoCard() {
22     // 卡片容器 (自带阴影, 圆角)

```

```

23     Card(
24         modifier = Modifier
25             .fillMaxWidth()
26             .padding(16.dp)
27             .clickable { println("卡片被点击") }, // 卡片可点击
28         shape = RoundedCornerShape(12.dp) // 卡片圆角12dp
29     ) {
30         // 卡片内部垂直布局
31         Column(
32             modifier = Modifier.padding(20.dp),
33             horizontalAlignment = Alignment.CenterHorizontally // 所有子元素水平
居中
34         ) {
35             // 圆形头像
36             Text(
37                 text = "小A",
38                 modifier = Modifier
39                     .size(100.dp)
40                     .background(Color(76, 217, 100), shape = CircleShape)
41                     .padding(16.dp),
42                 style = TextStyle(
43                     color = Color.White,
44                     fontSize = 24.sp,
45                     fontWeight = FontWeight.Bold
46                 )
47             )
48
49             Spacer(modifier = Modifier.height(16.dp)) // 空白间隔
50
51             // 用户名 (标题样式)
52             Text(
53                 text = "Android小白",
54                 style = TextStyle(
55                     fontSize = 20.sp,
56                     fontWeight = FontWeight.Bold
57                 )
58             )
59
60             Spacer(modifier = Modifier.height(8.dp))
61
62             // 个人简介 (正文样式)
63             Text(
64                 text = "正在学习Jetpack Compose, 一起进步!",
65                 style = TextStyle(
66                     fontSize = 16.sp,
67                     color = Color.Gray
68                 )

```

```
69         )
70     }
71 }
72 }
```

运行这段代码，你会得到一个美观、有层次的个人信息卡片，这就是今天知识点的综合应用！

五、本章小结（小白必背）

1. Column是垂直布局容器，用verticalArrangement控制垂直分布，horizontalAlignment控制水平对齐。
2. 修饰符（Modifier）是UI的"化妆师"，支持链式调用，常用的有padding、background、size、clickable。
3. 颜色用Color类，支持系统色、主题色、自定义色；文字用sp单位，排版用TextStyle。
4. 形状用Shape类，常用CircleShape（圆形）和RoundedCornerShape（圆角矩形），让UI更美观。

 课后作业：修改"个人信息卡片"的代码，比如把头像颜色换成自己喜欢的，把文字改成自己的信息，给卡片加一个边框。动手修改才能真正掌握！

（注：文档部分内容可能由 AI 生成）